Technology Stack

Phase 2: Requirement Analysis

1. Stack Overview

trvale-book separates the browser interface from the API service. The frontend handles the wizard, state and presentation; the backend validates requests, coordinates AI generation, calculates budgets and exposes trip operations; Supabase provides authentication and the database.

Layer	Technology (from package.json / requirements.txt)	Role
Frontend	React 19, Vite 8, Tailwind CSS 3.4, React Router 7, Axios, lucide-react, canvas-confetti	Pages, components, routing, styling, HTTP client
Frontend auth and data	@supabase/supabase-js 2.x	Sign-in, sign-up, session token
Frontend tests	Vitest 3	Unit tests for formatters and sample data
Backend	Python 3.11, FastAPI, Uvicorn	REST API and routing
Validation and settings	Pydantic 2, pydantic-settings, python-dotenv	Request/response schemas, environment configuration
HTTP and AI calls	httpx	Calls to Gemini or OpenAI
Database and auth	Supabase (PostgreSQL, Auth, RLS), supabase-py	Persistence and security
Backend tests	pytest, pytest-asyncio	API, validation, budget and AI tests
Hosting	Vercel (frontend), Render (backend)	Deployment

2. Frontend: React, Vite and Tailwind CSS

React provides the component-based interface. Context providers hold authentication and trip state. Vite supplies the development server and production build, and Tailwind supplies utility-class styling, including a print stylesheet that hides non-essential interface parts. The project includes a vercel.json rewrite so that client-side routes such as /plan or /trips load correctly on Vercel.

3. Backend: Python and FastAPI

FastAPI exposes the health, trips and export routes and generates interactive API documentation at /docs. Pydantic models define request and response shapes. The AI service, budget service and export service are separate modules so route handlers stay short.

4. Generative AI: Gemini or OpenAI

The AI provider is selected with the AI_PROVIDER setting. The default is Google Gemini (model name gemini-1.5-flash through AI_MODEL_NAME); OpenAI is supported through the chat completions endpoint with gpt-4o-mini. The prompt asks for strictly valid JSON, and the reply is parsed and hydrated into the application's trip structure. When no key is configured or a call fails, a heuristic generator is used. Model output must always be treated as untrusted until validated.

5. Database and Security: Supabase

A version-controlled SQL migration creates the profiles, trips, trip_preferences, itinerary_days and itinerary_activities tables, indexes, updated-at triggers, an automatic profile-creation trigger for new users, and row-level security policies so that users can access only rows where auth.uid() matches the owner.

6. Hosting and Version Control

The frontend is deployed on Vercel at https://travel-book-livid.vercel.app. A render.yaml and Procfile are included for deploying the backend on Render. Separate hosting needs a correct VITE_API_BASE_URL and a CORS_ORIGINS list that includes the deployed frontend origin. Git is used for version control; secret values must never be committed.

7. Configuration

Environment variable names are listed in .env.example files for the root, frontend and backend. Real values belong in private environment settings. The frontend needs VITE_API_BASE_URL and, for live authentication, VITE_SUPABASE_URL and VITE_SUPABASE_ANON_KEY. The backend needs the AI key, SUPABASE_URL, SUPABASE_KEY and CORS_ORIGINS.